Implantação no Google Cloud

Documentação técnica - Prumo

> **Situação:** plano de implantação. Nada desta página está implementado no código ainda; hoje o backend grava arquivos no Supabase ou em `backend/uploads`. A execução da infraestrutura é da equipe que administra o Google Cloud.

Este documento descreve a implantação recomendada para armazenar e distribuir PDFs e vídeos do Prumo com segurança, desempenho e controle de custos.

1. Arquitetura recomendada

```
Administrador -- upload direto assinado --> Cloud Storage privado
                                                    |
Usuário -> Frontend -> Backend/Cloud Run             |
                         |                           |
                         +- valida usuário e setor   |
                         +- gera URL assinada        |
                                    |                |
                                    v                |
                              Cloud CDN <------------+
                                    |
                                    v
                              PDF ou vídeo
```

Use:

- Cloud Storage Standard para os arquivos atuais;
- bucket privado em `southamerica-east1`;
- Cloud CDN para arquivos acessados por muitos usuários;
- URLs assinadas para autorização temporária;
- Cloud Run com uma conta de serviço própria;
- Secret Manager para segredos;
- PostgreSQL somente para metadados e chaves dos objetos.

Não coloque PDFs ou vídeos no PostgreSQL, no GitHub ou no sistema de arquivos do Cloud Run.

2. Pré-requisitos

Solicite ao administrador do Google Cloud:

- ID do projeto;
- acesso ao faturamento do projeto;
- permissão para criar bucket, conta de serviço, balanceador e CDN;
- domínio que será usado, por exemplo `arquivos.empresa.com.br`;
- confirmação de que `southamerica-east1` atende às regras internas de localização de dados;
- orçamento mensal e contatos que receberão os alertas.

Instale e autentique o Google Cloud CLI:

```
gcloud auth login
gcloud config set project SEU_PROJECT_ID
```

Defina estes nomes antes de continuar:

```
PROJECT_ID=seu-projeto
REGION=southamerica-east1
BUCKET=seu-projeto-manuais-prod
SERVICE_ACCOUNT=manual-operacional-api
BACKEND_BUCKET=manual-operacional-files
DOMAIN=arquivos.empresa.com.br
```

O nome do bucket precisa ser globalmente exclusivo.

3. Ativar as APIs

```
gcloud services enable \
  storage.googleapis.com \
  compute.googleapis.com \
  run.googleapis.com \
  secretmanager.googleapis.com \
  artifactregistry.googleapis.com \
  logging.googleapis.com \
  monitoring.googleapis.com
```

4. Criar o bucket privado

```
gcloud storage buckets create gs://SEU_BUCKET \
  --project=SEU_PROJECT_ID \
  --location=southamerica-east1 \
  --default-storage-class=STANDARD \
  --uniform-bucket-level-access \
  --public-access-prevention
```

Confirme a configuração:

```
gcloud storage buckets describe gs://SEU_BUCKET
```

O resultado deve indicar:

- região `SOUTHAMERICA-EAST1`;
- classe `STANDARD`;
- acesso uniforme ativado;
- prevenção de acesso público aplicada.

Não conceda `roles/storage.objectViewer` para `allUsers` ou `allAuthenticatedUsers`.

5. Configurar CORS

Crie temporariamente um arquivo `cors.json`:

```
[
  {
    "origin": ["https://sistema.empresa.com.br"],
    "method": ["GET", "HEAD", "PUT", "OPTIONS"],
    "responseHeader": ["Content-Type", "Content-Length", "Range", "Content-Range", "ETag"],
    "maxAgeSeconds": 3600
  }
]
```

Aplique ao bucket:

```
gcloud storage buckets update gs://SEU_BUCKET --cors-file=cors.json
```

Inclua `http://localhost:5173` somente em um bucket ou ambiente de desenvolvimento. Não libere origem com `*` em produção.

6. Criar a identidade do backend

```
gcloud iam service-accounts create manual-operacional-api \
  --display-name="API do Prumo"
```

Conceda acesso aos objetos somente nesse bucket:

```
gcloud storage buckets add-iam-policy-binding gs://SEU_BUCKET \
  --member="serviceAccount:manual-operacional-api@SEU_PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/storage.objectAdmin"
```

Em produção, não baixe um arquivo JSON com chave da conta de serviço. Anexe a conta ao Cloud Run e use Application Default Credentials.

7. Definir a organização dos objetos

Use nomes imutáveis e versionados:

```
manuals/{manualId}/v{version}/documento-{uuid}.pdf
manuals/{manualId}/v{version}/video-{uuid}.mp4
temporary/{userId}/{uuid}.upload
```

Grave no PostgreSQL apenas a chave:

```
manuals/152/v2/documento-a1b2c3.pdf
```

Não grave:

- URL assinada;
- token de acesso;
- credencial do Google;
- conteúdo binário.

Uma URL assinada expira; por isso ela deve ser gerada quando o usuário solicita o arquivo.

8. Metadados dos arquivos

Ao finalizar o upload, defina:

Para PDF:

```
Content-Type: application/pdf
Content-Disposition: inline
Cache-Control: public, max-age=86400
```

Para vídeo:

```
Content-Type: video/mp4
Content-Disposition: inline
Cache-Control: public, max-age=86400
```

Como cada versão possui um caminho diferente, é possível aumentar futuramente o `max-age` sem servir uma versão antiga no lugar da atual.

9. Alterações necessárias no projeto

O projeto atual ainda usa Supabase ou armazenamento local em `backend/src/config/storage.js`. O upload já não fica na memória do processo: o arquivo vai para a pasta temporária do sistema e é apagado ao fim da requisição. No Cloud Run essa pasta também ocupa memória da instância, e o Cloud Run recusa requisições maiores que 32 MiB, então vídeos grandes só funcionam com o upload direto descrito abaixo.

A migração deve ser feita em duas etapas.

9.1 Upload direto

Crie no backend um endpoint semelhante a:

```
POST /api/uploads/sign
```

Entrada:

```
{
  "manualId": 152,
  "version": 2,
```

```
  "type": "pdf",
  "contentType": "application/pdf"
}
```

O backend deve:

1. autenticar o usuário;
2. exigir um perfil autorizado;
3. validar tipo e tamanho esperado;
4. criar uma chave imprevisível;
5. gerar URL assinada de `PUT` com validade de aproximadamente 10 minutos;
6. retornar a URL e a chave do objeto.

O frontend envia o arquivo diretamente ao Cloud Storage. O vídeo não deve passar pela memória do processo Node.js.

Depois do upload, use outro endpoint para confirmar que o objeto existe, conferir tamanho e `Content-Type`, e somente então gravar a chave no manual.

9.2 Leitura autorizada

Quando o usuário abre um manual, o backend deve:

1. consultar o manual;
2. validar perfil, setor e status ativo;
3. gerar uma URL assinada do CDN;
4. retornar a URL temporária ao frontend.

Validade sugerida:

- PDF: 10 minutos;
- vídeo: duração do vídeo mais 15 minutos;
- nunca usar URL sem expiração.

10. Instalar a biblioteca no backend

Na implementação do Cloud Storage:

```
cd backend
npm install @google-cloud/storage
```

Variáveis não secretas sugeridas:

```
GCS_BUCKET=seu-projeto-manuais-prod
GCS_REGION=southamerica-east1
FILES_BASE_URL=https://arquivos.empresa.com.br
UPLOAD_URL_TTL_SECONDS=600
PDF_URL_TTL_SECONDS=600
```

No Cloud Run, o projeto e a identidade são detectados automaticamente. Para desenvolvimento local, prefira `gcloud auth application-default login`; não versione credenciais.

11. Configurar Cloud CDN

Para muitos acessos, coloque um Application Load Balancer externo global e o Cloud CDN na frente do bucket.

11.1 Criar o backend bucket

```
gcloud compute backend-buckets create manual-operacional-files \
  --gcs-bucket-name=SEU_BUCKET \
  --enable-cdn
```

11.2 Criar uma chave de assinatura

Gere 128 bits aleatórios em base64url e armazene o valor no Secret Manager. Não coloque a chave no Git.

Adicione a chave ao backend bucket conforme o procedimento oficial de URLs assinadas do Cloud CDN. A adição da primeira chave também cria a identidade de preenchimento do cache.

11.3 Autorizar o preenchimento do cache

Descubra o número do projeto:

```
gcloud projects describe SEU_PROJECT_ID --format="value(projectNumber)"
```

Depois de adicionar a chave do CDN:

```
gcloud storage buckets add-iam-policy-binding gs://SEU_BUCKET \
  --member="serviceAccount:service-SEU_PROJECT_NUMBER@cloud-cdn-fill.iam.gserviceaccount.com" \
  --role="roles/storage.objectViewer"
```

Esse acesso permite que o CDN leia o bucket privado. Não torna os objetos públicos.

11.4 Finalizar o balanceador

No console do Google Cloud:

1. acesse **Network Services > Load balancing**;
2. crie um Application Load Balancer externo global;
3. use o backend bucket criado anteriormente;
4. reserve um IP global estático;
5. associe `arquivos.empresa.com.br` ao IP;
6. configure certificado gerenciado pelo Google;
7. force HTTPS;
8. habilite logs do balanceador e do CDN.

As instruções exatas de URL map, proxy e forwarding rule podem variar conforme o padrão de rede da empresa. A equipe de infraestrutura deve aplicar as políticas corporativas existentes.

Documentação oficial: <https://docs.cloud.google.com/cdn/docs/using-signed-urls>

12. Implantar o backend no Cloud Run

Use a conta criada como identidade do serviço:

```
gcloud run deploy manual-operacional-api \
  --source=backend \
  --region=southamerica-east1 \
  --service-account=manual-operacional-api@SEU_PROJECT_ID.iam.gserviceaccount.com \
  --set-env-vars="GCS_BUCKET=SEU_BUCKET,GCS_REGION=southamerica-east1,FILES_BASE_URL=https://arquivos.emp
resa.com.br"
```

Coloque `DATABASE_URL`, `JWT_SECRET` e a chave de assinatura do CDN no Secret Manager. Não passe segredos em `--set-env-vars`.

Documentação oficial: <https://docs.cloud.google.com/run/docs/configuring/services/secrets>

12.1 Capacidade para muitos acessos simultâneos

O backend não guarda estado na memória que precise ser compartilhado (a sessão é o JWT e os setores vêm do banco a cada requisição), então pode rodar em várias instâncias. Pontos a combinar com a equipe de infraestrutura:

- **Conexões com o banco:** cada instância abre até `DB_POOL_MAX` conexões. O número máximo de instâncias multiplicado por `DB_POOL_MAX` precisa ficar abaixo do `max_connections` do PostgreSQL, com folga para administração. Exemplo: 10 instâncias x 10 conexões = 100.
- **Instâncias mínimas:** mantenha pelo menos 1 instância ativa (`--min-instances=1`) no horário de uso, para o primeiro acesso não esperar a inicialização.

- **Health checks:** `GET /health` para vida e `GET /health/ready` para prontidão (consulta o banco).
- **Desligamento:** o backend trata `SIGTERM` e termina as requisições em andamento antes de sair.
- **Arquivos:** com mais de uma instância, `backend/uploads` não funciona, porque cada instância tem a própria pasta. Use o Cloud Storage desta página antes de escalar.
- **Bloqueio de login:** as tentativas erradas ficam na memória de cada instância. Com N instâncias, o limite efetivo pode chegar a 5 x N tentativas antes do bloqueio. Se isso for um problema, mova o controle para o banco ou para o Cloud Armor.
- **IP real do usuário:** `TRUST_PROXY_HOPS=1` atende ao Cloud Run acessado direto. Com um Application Load Balancer externo na frente, use 2.

13. Regras de ciclo de vida

Arquivos atuais e frequentemente acessados devem continuar em Standard.

Política inicial sugerida:

- remover uploads temporários não confirmados após 1 dia;
- mover versões antigas para Nearline somente depois de 90 dias;
- mover versões antigas para Coldline depois de 365 dias, se a política corporativa permitir;
- não excluir documentos antes do prazo definido pela empresa;
- não aplicar Nearline/Coldline ao vídeo atual.

Antes de criar exclusões automáticas, valide retenção legal, auditoria e recuperação com Segurança da Informação.

14. Orçamento e monitoramento

No Billing, crie um orçamento para o projeto com alertas em:

- 50%;
- 80%;
- 100%;
- 120% do valor mensal planejado.

Monitore:

- bytes entregues pelo CDN;
- taxa de acerto do cache;
- preenchimento do cache;
- quantidade e tamanho dos objetos;
- respostas 403, 404 e 5xx;
- latência de download;
- uploads interrompidos;
- custo por serviço e SKU.

Para a América do Sul, a saída do Cloud CDN começa atualmente em aproximadamente US$ 0,09 por GiB nos primeiros 10 TiB. Consulte sempre a tabela atual antes da aprovação financeira: <https://cloud.google.com/cdn/pricing>.

15. Segurança

- mantenha Public Access Prevention ativado;
- use uma conta de serviço exclusiva por ambiente;
- aplique menor privilégio;
- separe buckets de desenvolvimento, homologação e produção;
- valide extensão, MIME type e assinatura real do arquivo;

- limite tamanho e quantidade de uploads;
- registre criação, substituição e exclusão no audit log;
- use nomes imprevisíveis;
- não exponha a chave original do objeto a usuários não autorizados;
- rotacione a chave de assinatura do CDN;
- avalie malware scanning conforme a política interna;
- configure retenção e recuperação antes de ativar exclusões.

16. Testes de aceitação

Antes de liberar produção, confirme:

- [ ] bucket não aceita leitura anônima;
- [ ] administrador autorizado consegue enviar PDF e vídeo;
- [ ] usuário comum não consegue gerar URL para manual de outro setor;
- [ ] manual inativo não gera URL;
- [ ] URL expirada retorna 403;
- [ ] PDF abre no navegador;
- [ ] vídeo permite avançar e retroceder;
- [ ] segundo acesso ao mesmo arquivo apresenta `cache hit` nos logs;
- [ ] arquivos grandes não passam pela memória do Cloud Run;
- [ ] troca de versão mantém o histórico;
- [ ] exclusão de manual segue a política de retenção;
- [ ] alertas de orçamento chegam aos responsáveis;
- [ ] logs não expõem tokens ou URLs assinadas completas.

17. Ordem recomendada de execução

1. Aprovar região, orçamento, domínio e retenção.
2. Criar bucket privado de homologação.
3. Criar conta de serviço e permissões mínimas.
4. Implementar upload direto assinado.
5. Implementar confirmação do upload.
6. Implementar leitura autorizada.
7. Testar sem CDN em homologação.
8. Criar Load Balancer e Cloud CDN.
9. Implementar assinatura de URLs do CDN.
10. Configurar domínio e certificado.
11. Criar monitoramento, orçamento e alertas.
12. Executar os testes de aceitação.
13. Migrar arquivos existentes.
14. Liberar produção gradualmente.
15. Remover Supabase e armazenamento local somente após validar a migração.

18. Estratégia de migração

O sistema ainda não tem banco nem arquivos em produção. Esta seção só se aplica se algum ambiente com dados reais (Supabase ou pasta local) for criado antes da migração para o Google Cloud.

Durante a transição, mantenha compatibilidade com os três formatos:

```
https://...        arquivo antigo no Supabase
/uploads/...       arquivo local antigo
manuals/...        nova chave no Cloud Storage
```

Crie um processo de migração que:

1. leia o arquivo antigo;
2. envie ao bucket privado;
3. valide tamanho e hash;
4. atualize o banco com a chave nova;
5. teste a abertura;
6. registre a migração;
7. remova o arquivo antigo somente após o período de segurança.

Não faça uma substituição total sem inventário, backup e possibilidade de retorno.

Referências oficiais

- Cloud Storage pricing: <https://cloud.google.com/storage/pricing>
- Cloud CDN pricing: <https://cloud.google.com/cdn/pricing>
- Cloud Storage signed URLs: <https://docs.cloud.google.com/storage/docs/access-control/signed-urls>
- Cloud CDN signed URLs: <https://docs.cloud.google.com/cdn/docs/using-signed-urls>
- Cloud CDN private bucket access: <https://docs.cloud.google.com/cdn/docs/configure-private-bucket-access>
- Cloud Run service identity: <https://docs.cloud.google.com/run/docs/configuring/services/service-identity>
- Cloud Run secrets: <https://docs.cloud.google.com/run/docs/configuring/services/secrets>